



FAST- National University of Computer & Emerging Sciences, Karachi.

School of Computing,

Final Examination, Spring 2023

22nd May 2023, 12:00 pm – 03:00 pm



Course Code: CS-2001	Course Name: Data Structures
Instructors: Anam Qureshi, Safia Baloch, Nida Munawwar, and Mafaza Mohi	
Student Roll No:	Section:

Time Allowed: 180 minutes

Maximum Points: 50

Graph Data Structure	
Question No. 1 [CLO 4]	[Points: 8]

	A	B	C	D	E
A	0	3	1	∞	∞
B	3	0	5	6	7
C	1	5	0	2	4
D	∞	6	2	0	8
E	∞	7	4	8	0

Suppose you are working for a construction company that has been tasked with building a new road network in a remote area. The company wants to build the road network to minimize the total cost while ensuring that all areas are connected to the network. To accomplish this, they want you to design a solution that will provide them with minimum cost. Apply the proposed solution to the given adjacency matrix.

Solution:

1. Create a class to represent the graph data structure, including nodes and edges.
2. Create a hard-coded adjacency matrix that contains the distances between each pair of locations.
3. Assign each location an index that corresponds to its row and column position in the adjacency matrix.
4. Create a class to represent a node, including its name, index, and distance from the source node (initialized as infinity).
5. Create a class to represent an edge, including its weight and the source and destination

nodes.

6. Create a priority queue to store nodes, ordered by their distance from the source node.
7. Create an array to store the shortest path distances from the source node to each node in the graph.
8. Create an array to store the previous node in the shortest path to each node in the graph.
9. Initialize the shortest path distance array with infinity for all nodes except the source node, which is initialized to 0.
10. Add the source node to the priority queue.
11. While the priority queue is not empty, do the following:
 - a. Get the node with the smallest distance from the priority queue.
 - b. For each of its adjacent nodes, calculate the distance to the adjacent node through the current node.
 - c. If this distance is shorter than the current shortest path distance to the adjacent node, update the shortest path distance and previous node arrays and add the adjacent node to the priority queue.
12. Once the shortest path distances have been calculated, traverse the previous node array from the destination node to the source node to find the shortest path.
13. Output the shortest path and estimated travel time (sum of the edge weights).

Here's a sample implementation in Java:

```
import java.util.Scanner;
```

```
public class CityGraph {
```

```
    private static final int INFINITY = Integer.MAX_VALUE;
```

```
    private int[][] adjacencyMatrix;
```

```
    private int numVertices;
```

```
    public CityGraph(int[][] adjacencyMatrix) {
```

```
        this.adjacencyMatrix = adjacencyMatrix;
```

```
        numVertices = adjacencyMatrix.length;
```

```
    }
```

```
    public void dijkstra(int source, int destination) {
```

```
        int[] shortestDistances = new int[numVertices];
```

```
        boolean[] visited = new boolean[numVertices];
```

```
        for (int i = 0; i < numVertices; i++) {
```

```
            shortestDistances[i] = INFINITY;
```

```
            visited[i] = false;
```

```
        }
```

```

shortestDistances[source] = 0;

for (int i = 0; i < numVertices - 1; i++) {
    int minDistance = INFINITY;
    int closestVertex = -1;

    for (int j = 0; j < numVertices; j++) {
        if (!visited[j] && shortestDistances[j] < minDistance) {
            minDistance = shortestDistances[j];
            closestVertex = j;
        }
    }

    visited[closestVertex] = true;

    for (int k = 0; k < numVertices; k++) {
        int edgeDistance = adjacencyMatrix[closestVertex][k];

        if (edgeDistance > 0 && shortestDistances[closestVertex] + edgeDistance <
            shortestDistances[k]) {
            shortestDistances[k] = shortestDistances[closestVertex] + edgeDistance;
        }
    }
}

if (shortestDistances[destination] == INFINITY) {
    System.out.println("There is no path between the selected locations.");
} else {
    System.out.println("Shortest path from " + vertexName(source) + " to " +
        vertexName(destination) + " is:");
    printPath(destination, shortestDistances);
    System.out.println("Estimated travel time: " + shortestDistances[destination] +
        " minutes.");
}

private void printPath(int vertex, int[] distances) {

```

```

if (vertex < 0 || vertex >= numVertices) {
    throw new IllegalArgumentException("Invalid vertex.");
}

```

```

if (distances[vertex] == INFINITY) {
    System.out.print(vertexName(vertex));
} else if (vertex == 0) {

```

```

    System.out.print(vertexName(vertex));
} else {
    printPath(getPredecessor(vertex, distances), distances);
    System.out.print(" -> " + vertexName(vertex));
}
}

```

```

private int getPredecessor(int vertex, int[] distances) {
    int minDistance = INFINITY;
    int predecessor = -1;

```

```

    for (int i = 0; i < numVertices; i++) {
        int edgeDistance = adjacencyMatrix[i][vertex];

```

```

        if (edgeDistance > 0 && distances[i] + edgeDistance < minDistance) {
            minDistance = distances[i] + edgeDistance;
            predecessor = i;
        }
    }

```

```

    return predecessor;
}

```

```

private String vertexName(int vertex) {
    return String.valueOf((char) ('A' + vertex));
}

```

```

public static void main(String[] args) {
    int[][] adjacencyMatrix = {
        {0, 5, 2, INFINITY},
        {5, 0, 1, 7},

```

```
{2, 1, 0, 3},
{INFINITY, 7, 3, 0}
};
```

```
CityGraph cityGraph = new CityGraph
```

Hashing and Balanced Binary Search Tree	
Question No. 2 [CLO 3]	[Points: 7]

Grace is implementing a program for the popular word game Scrabble. She is considering the use of a hashtable (using separate chaining) or a balanced binary search tree to store the dictionary of legal words with their definitions. For the hash table, she would use a hash function that adds up the ASCII values of all of the letters in the word. Whereas, the balanced binary search tree is ordered by alphabetical ordering. Which data structure, hash table or balanced binary search tree, would allow her to find all words that contain AERST once more easily? Explain your choice. Also, write an algorithm for the justification of your choice.

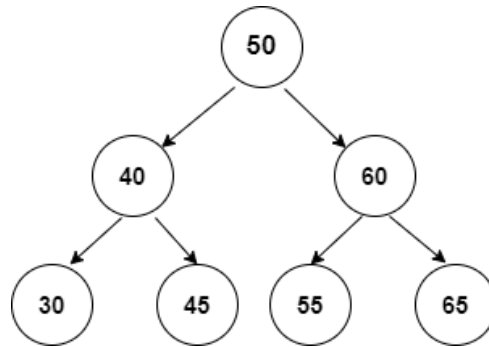
Solution:

For the given hash function, a hash table would put all the words that contain AERST once in the same chain, making them easier to find.

Students must write an algorithm of hashing using any collision detection technique.

Binary Search Tree	
Question No. 3 [CLO 2]	[Points:7]

Ahmed and Ali are playing a unique game, there are seven balls on the snooker table, and each opponent has consecutive three turns to hit the balls forming the largest trio. It's Ali's turn to have three consecutive turns. Write a program that will help him to find the largest trio of balls. The balls are arranged in the following manner.



Solution:

A simple solution is to traverse the binary tree in a preorder fashion and for each encountered node,

check whether the subtree rooted at the node is a BST or not. If the subtree is a BST, calculate and return

the subtree's size rooted at the node. Otherwise, return the maximum size BST returned by the left and

right subtrees.

```
include <iostream>
```

```
#include <limits>
```

```
using namespace std;
```

```
// Data structure to store a BST node
```

```
struct Node
```

```
{
```

```
// stores value of this node
```

```
int data;
```

```
// stores left and right child pointer for this node
```

```
Node *left, *right;
```

```
// constructor
```

```
Node(int data)
```

```
{
```

```
this->data = data;
```

```
this->left = this->right = nullptr;
```

```
}
```

```
};
```

```
// Data structure to store information about a binary tree
```

```
struct SubTreeInfo
```

```
{
```

```
// stores the minimum and the maximum value in the binary tree rooted under the  
// current node. They are relevant only if the `isBST` flag is true
```

```
int min, max;
```

```
// stores size of the largest BST in the binary tree rooted under
```

```
// the current node
```

```
int size;
```

```
// true if a binary tree rooted under the current node is a BST
```

```

bool isBST;
// constructor
SubTreeInfo(int min, int max, int size, bool isBST)
{
    this->min = min;
    this->max = max;
    this->size = size;
    this->isBST = isBST;
}
};
// Recursive function to find the size of the largest BST in a given binary tree
SubTreeInfo* findLargestBST(Node* root)
{
    // Base case: empty tree
    if (root == nullptr) {
        return new SubTreeInfo(INT_MAX, INT_MIN, 0, true);
    }
    // Recur for the left and right subtrees
    SubTreeInfo* left = findLargestBST(root->left);
    SubTreeInfo* right = findLargestBST(root->right);
    SubTreeInfo* info = nullptr;
    // Check if a binary tree rooted under the current root is a BST
    // 1. Left and right subtree are also BST
    // 2. The value of the root node should be more than the largest value
    // in the left subtree
    // 3. The value of the root node should be less than the smallest value
    // in the right subtree
    if (left->isBST && right->isBST &&
        (root->data > left->max && root->data < right->min))
    {
        info = new SubTreeInfo(min(root->data, min(left->min, right->min)),
            max(root->data, max(left->max, right->max)),
            left->size + 1 + right->size,
            true);
    }
    else {
        // If a binary tree rooted under the current root is not a BST,
        // return the largest BST size in its left and right subtree
        info = new SubTreeInfo(0, 0,
            max(left->size, right->size),
            false);
    }
    // deallocate the memory for the left and right subtrees info node
    delete(left), delete(right);
    return info;
}

```

```

}
int main()
{
/* Construct the following binary tree
10

/\
/\
15 8
/>\
/>\
12 20 5 9
/>\
/>\
2 14 4 7 10
*/
Node* root = new Node(10);
root->left = new Node(15);
root->right = new Node(8);
root->left->left = new Node(12);
root->left->right = new Node(20);
root->right->left = new Node(5);
root->right->right = new Node(9);
root->left->left->left = new Node(2);
root->left->left->right = new Node(14);
root->right->left->left = new Node(4);
root->right->left->right = new Node(7);
root->right->right->right = new Node(10);
cout << "The size of the largest BST is " <<
findLargestBST(root)->size;

return 0;
}

```

Binary Heap/ Priority Queue	
Question No. 4 [CLO 1]	[Points:7]

You are building a call center application that handles customer calls. Each call has a priority assigned to it based on its importance. Design a priority queue to efficiently manage incoming calls and handle them based on priority. Implement the following operations:
 Enqueue_call (call, priority): Add a call with the given priority to the queue
 Dequeue_call (): Retrieve and remove the highest priority call from the queue

SOLution:


```

include <iostream>
include <queue>
include <string>

struct Call {
    std::string call;
    int priority;

    bool operator<(const Call& other) const {
        return priority < other.priority;
    }
};

class CallCenter {
private:
    std::priority_queue<Call> callQueue;

public:
    void Enqueue_call(const std::string& call, int priority) {
        Call newCall;
        newCall.call = call;
        newCall.priority = priority;
        callQueue.push(newCall);
        std::cout << "Call enqueued: " << call << ", Priority: " <<
priority << std::endl;
    }

    void Dequeue_call() {
        if (!callQueue.empty()) {
            Call highestPriorityCall = callQueue.top();
            callQueue.pop();
            std::cout << "Dequeued call: " << highestPriorityCall.call <<
", Priority: " << highestPriorityCall.priority << std::endl;
        } else {
            std::cout << "Call queue is empty!" << std::endl;
        }
    }
};

int main() {
    CallCenter center;

    center.Enqueue_call("Call 1", 3);
    center.Enqueue_call("Call 2", 1);
    center.Enqueue_call("Call 3", 2);

    center.Dequeue_call();
    center.Dequeue_call();
    center.Dequeue_call();
    center.Dequeue_call();

    return 0;
}

```

}

Stack and Queues	
Question No. 5 [CLO 1]	[Points:7]

Design a program that collects daily price quotes for some stock and returns the span of the stock's price for the current day. The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal to 2 for 4 consecutive days.

Also, if the prices of the stock in the last four days is [7,3,4,1,2] and the price of the stock today is, then the span of today is 3 because starting from today, the price of the stock was less than or equal to 8 for 3 consecutive days.

Solution:

```
include <iostream>
include <stack>

void calculateSpan(int prices[], int span[], int size) {
    std::stack<int> stack;
    stack.push(0);

    span[0] = 1;

    for (int i = 1; i < size; i++) {
        while (!stack.empty() && prices[i] >= prices[stack.top()]) {
            stack.pop();
        }

        span[i] = stack.empty() ? i + 1 : i - stack.top();
        stack.push(i);
    }
}

int main() {
    int prices[] = {7, 2, 1, 2};
    int size = sizeof(prices) / sizeof(prices[0]);
    int span[size];

    calculateSpan(prices, span, size);

    std::cout << "Span of stock's price for the current day: ";
    for (int i = 0; i < size; i++) {
        std::cout << span[i] << " ";
    }
    std::cout << std::endl;
```

```
    return 0;
}
```

Advanced Sorting Techniques and Linked List	
Question No. 6 [CLO 3]	[Points:7]

You work for a social media company that allows users to create and share playlists of songs. Each playlist is represented as a linked list of songs, where each song is represented by a node in the linked list. Users can follow other users and view their playlists. To improve the user experience, the company wants to implement a feature that merges two or more playlists of a user and his followed users into one playlist that is sorted by the song title.

Write a program that takes an array of linked lists representing the playlists to merge and return a sorted linked list containing all the songs from the original playlists. Each song is represented by a node in the linked list with the following properties: song title, artist name, and duration in seconds. The linked list should be sorted by song title in ascending order.

Solution:

```
public class Node {
    public String title;
    public String artist;
    public int duration;

    public Node(String title, String artist, int duration) {
        this.title = title;
        this.artist = artist;
        this.duration = duration;
    }
}

public class LinkedList {
    public Node head;

    public LinkedList() {
        head = null;
    }

    public void add(Node node) {
        if (head == null) {
            head = node;
        } else {
            Node curr = head;
            while (curr.next != null) {
```

```
curr = curr.next;  
}
```

```
curr.next = node;  
}  
}  
}
```

```
public class Main {  
    public static LinkedList mergePlaylists(LinkedList[] playlists) {  
        LinkedList mergedList = new LinkedList();
```

```
        // Find the smallest node in all playlists and add it to the merged list  
        while (true) {  
            boolean found = false;  
            Node smallest = null;  
            int smallestIndex = -1;  
            for (int i = 0; i < playlists.length; i++) {  
                LinkedList playlist = playlists[i];  
                if (playlist != null && playlist.head != null) {  
                    Node curr = playlist.head;  
                    if (smallest == null || curr.title.compareTo(smallest.title) < 0) {  
                        smallest = curr;  
                        smallestIndex = i;  
                        found = true;  
                    }  
                }  
            }  
            if (!found) {  
                break;  
            }  
            mergedList.add(new Node(smallest.title, smallest.artist, smallest.duration));  
            playlists[smallestIndex].head = smallest.next;  
        }
```

```
        return mergedList;  
    }
```

```
    public static void main(String[] args) {  
        LinkedList list1 = new LinkedList();  
        list1.add(new Node("Song 1", "Artist 1", 180));  
        list1.add(new Node("Song 2", "Artist 2", 210));  
        list1.add(new Node("Song 3", "Artist 3", 150));
```

```
        LinkedList list2 = new LinkedList();  
        list2.add(new Node("Song 1", "Artist 1", 180));
```

```
list2.add(new Node(&quot;Song 4&quot;, &quot;Artist 4&quot;, 240));
list2.add(new Node(&quot;Song 5&quot;, &quot;Artist 5&quot;, 300));
```

```
LinkedList list3 = new LinkedList();
list3.add(new Node(&quot;Song 2&quot;, &quot;Artist 2&quot;, 210));
list3.add(new Node(&quot;Song 3&quot;, &quot;Artist 3&quot;, 150));
list3.add(new Node(&quot;Song 6&quot;, &quot;Artist 6&quot;, 180));
```

```
LinkedList[] playlists = new LinkedList[] { list1, list2, list3 };
```

```
LinkedList mergedList = mergePlaylists(playlists);
Node curr = mergedList.head;
while (curr != null) {
System.out.println(curr.title + &quot; - &quot; + curr.artist + &quot; - &quot; + curr.duration +
&quot;s&quot;);
curr = curr.next;
}
}
}
```

Recursion and Backtracking	
Question No. 7 [CLO 2]	[Points:7]

You are given an integer array of cookies, where `cookies[i]` denotes the number of cookies in the i^{th} bag. You are also given an integer `k` that denotes the number of children to distribute all the bags of cookies to. All the cookies in the same bag must go to the same child and cannot be split up.

The unfairness of a distribution is defined as the maximum number of total cookies obtained by a single child in the distribution. Write a recursive program that will return the minimum unfairness of all distributions.

For Example:

Input: `cookies = [8,15,10,20,8]`, `k=2`

Output: 31

Explanation: One optimal distribution is `[8,15,8]` and `[10,20]`

- The first child receives `[8,15,8]` which has a total of $8+15+8=31$ cookies
- The second child receives `[10,20]` which has a total of $10+20=30$ cookies

The unfairness of the distribution is $\max(31,30)=31$.

Solution:

```
include <iostream>
include <vector>
include <climits>
```

```

int distributeCookies(const std::vector<int>& cookies, int k, int index,
int maxSoFar) {
    if (k == 1) {
        int sum = 0;
        for (int i = index; i < cookies.size(); i++) {
            sum += cookies[i];
        }
        return std::max(maxSoFar, sum);
    }

    int minUnfairness = INT_MAX;
    int sum = 0;
    for (int i = index; i < cookies.size() - k + 1; i++) {
        sum += cookies[i];
        int maxUnfairness = std::max(maxSoFar, sum);
        int nextUnfairness = distributeCookies(cookies, k - 1, i + 1,
maxUnfairness);
        minUnfairness = std::min(minUnfairness, nextUnfairness);
    }
    return minUnfairness;
}

int findMinimumUnfairness(const std::vector<int>& cookies, int k) {
    return distributeCookies(cookies, k, 0, 0);
}

int main() {
    std::vector<int> cookies = {8, 15, 10, 20, 8};
    int k = 2;

    int minUnfairness = findMinimumUnfairness(cookies, k);

    std::cout << "Minimum unfairness: " << minUnfairness << std::endl;

    return 0;
}

```

***** Good Luck *****